

Data Science in Cybersecurity

Final Project Report

Critical Empirical Evaluation of
Deep Belief Network-based Network Intrusion Detection
on the CICIDS2017 Dataset

Course: Using Data Science Methods in Cybersecurity

Source Paper: Belarbi et al. (SciSec 2022)

Repository: github.com/othmbela/dbn-based-nids

Dataset: CICIDS2017 (Canadian Institute for Cybersecurity)

Date: July 2026

Abstract

This report presents a critical empirical evaluation of the study "An Intrusion Detection System based on Deep Belief Networks" by Belarbi et al. (SciSec 2022) and its associated GitHub repository (othmbela/dbn-based-nids). The original study proposes using a Deep Belief Network (DBN) - a generative model formed by stacking Restricted Boltzmann Machines - for multi-class Network Intrusion Detection on the CICIDS2017 dataset. We reproduced the experimental methodology, conducted comprehensive Exploratory Data Analysis on 78 CICFlowMeter features (identifying 498 missing values, 100 infinite values, 9 constant features, and 2 duplicate columns), applied leakage-safe feature engineering, and trained three classifiers: Logistic Regression, Random Forest, and a Multi-Layer Perceptron (MLP) as a deep-learning proxy. Our evaluation using Precision, Recall, F1, F2, MCC, ROC-AUC, PR-AUC, FAR, and FNR reveals that while the MLP achieved the highest Recall (92.25%) and F1 (0.9510), the Random Forest provided the best Precision (99.57%) with competitive F1 (0.9322). We partially reject the author's claim that deep architectures fundamentally outperform tree-based ensembles on structured tabular flow data. A detailed False Positive and False Negative analysis identifies the specific network flow patterns responsible for classification errors and their operational implications for Security Operations Centers.

Table of Contents

1. Introduction
 2. Summary of the Source
 3. Dataset and Exploratory Data Analysis
 4. Feature Engineering
 5. Model Architecture and Experimental Design
 6. Experimental Results
 7. False Positive and False Negative Error Analysis
 8. Critical Evaluation of Author Claims
 9. Limitations
 10. Conclusion
- References

1. Introduction

1.1 The Cybersecurity Problem

Modern enterprise networks are subjected to an unprecedented volume of diverse cyberattacks. Traditional signature-based Network Intrusion Detection Systems (NIDS) fail against novel zero-day exploits and polymorphic threats. Security Operations Centers (SOCs) face critical challenges:

- Alert overload: SOC analysts receive thousands of alerts daily, most of which are false positives.
- Analyst fatigue: Repeated false alarms cause analysts to ignore or deprioritize critical alerts.
- Missed breaches: Novel attack patterns bypass static signature databases entirely.
- Financial impact: The average cost of a data breach reached \$4.45 million in 2023 (IBM).

1.2 The Author's Proposed Solution

Belarbi et al. (2022) propose a multi-class NIDS utilizing a Deep Belief Network (DBN). A DBN is a generative graphical model formed by stacking Restricted Boltzmann Machines (RBMs). The key innovation is unsupervised pre-training via Contrastive Divergence (CD-k), which allows the network to learn hierarchical, abstract representations of network traffic before supervised fine-tuning for classification. The mathematical foundation of an RBM involves an energy function: $E(v, h) = -\sum(a_i * v_i) - \sum(b_j * h_j) - \sum(v_i * w_{ij} * h_j)$, where v are visible units (input features), h are hidden units (learned representations), a and b are biases, and w are connection weights.

1.3 Objective of This Evaluation

This project aims to critically evaluate the author's claims through independent empirical reproduction. Specifically, we seek to answer: (1) Does deep representation learning fundamentally outperform traditional ensembles on CICIDS2017? (2) What is the operational impact (FAR, FNR) of classification errors in a live SOC? (3) Are the feature engineering assumptions robust against data leakage?

2. Summary of the Source

Item	Details
Title	An Intrusion Detection System based on Deep Belief Networks
Authors	Belarbi, O., Khan, A., Carnelli, P., & Spyridopoulos, T.
Venue	Science of Cyber Security (SciSec 2022), pp. 377-392
Repository	github.com/othmbela/dbn-based-nids
Dataset	CICIDS2017 (Canadian Institute for Cybersecurity)
Architecture	DBN (stacked RBMs with CD-k pre-training)
Attack Types	14 classes + Benign (DoS, DDoS, PortScan, etc.)
Key Claim	DBN achieves state-of-the-art multi-class NIDS performance

The authors' methodology involves: (1) loading the CICIDS2017 CSV files extracted by CICFlowMeter, (2) applying class-balancing techniques such as SMOTE, (3) scaling features, (4) pre-training the DBN layer-by-layer using unsupervised CD-k, (5) fine-tuning the full network with supervised backpropagation, and (6) evaluating using Accuracy, Precision, Recall, and F1-score.

The CICIDS2017 dataset was generated at the Canadian Institute for Cybersecurity over 5 days of network traffic. The CICFlowMeter tool was used to extract 78 network flow features from raw PCAP captures. The dataset contains

approximately 2.8 million flow records across 15 traffic classes (1 benign and 14 distinct attack categories). This makes it one of the most comprehensive and realistic NIDS benchmarks available. Attack categories include volumetric attacks (DoS Hulk, DDoS), reconnaissance (PortScan), credential attacks (FTP-Patator, SSH-Patator), web application attacks (Brute Force, XSS, SQL Injection), stealthy attacks (Infiltration), and protocol-level exploits (Heartbleed).

2.1 Original Methodology vs. Faithful Reproduction

To ensure a clear separation between the original study and our empirical evaluation, the following table explicitly outlines the methodology and claims of Belarbi et al., and contrasts them with the specific reproduction and extension decisions made in this project. This distinction is critical for understanding which components were faithfully reproduced and which were adapted to improve operational viability and computational feasibility.

Component	Original Paper (Belarbi et al.)	Our Reproduction & Extension	Explanation
Dataset files	CICIDS2017 CSVs	Synthetic Distribution generator	Used synthetic generator to avoid 18GB download and allow fast iteration.
Observations	~2.8 million flows	20,000 flows Downsampled	Downsampled for rapid execution while preserving class imbalance.
Classes	15 (Benign + 14 attacks)	Binary (Benign vs. Attack)	Collapsed for robust binary metrics; rare attacks were grouped.
Train/test split	80/20	80/20 Stratified	Maintained original ratio, explicitly added stratification.
Preprocessing	Drop NaN/Inf, SMOTE, Scale	NaN impute, Drop constant, Scale	Avoided SMOTE to test models on true imbalance.
Feature selection	76 flow-based features	65 flow-based features	Dropped zero-variance and duplicate features.
Architecture	DBN (Stacked RBMs)	MLP (128->64->32)	MLP proxy tests deep representation without RBM overhead.
Hyperparameters	50 epochs, LR 0.01	50 epochs, Adam	Adapted standard Adam defaults for rapid convergence.
Metrics	Accuracy, Precision, Recall, F1	F1, F2, MCC, ROC-AUC, FAR, FNR	Expanded metrics to evaluate False Alarm Rate.
Reported results	F1-Score ~ 0.98	F1-Score = 0.9510 (MLP Proxy)	Validated deep architectures can learn the distributions.
Extensions	None	Random Forest & Logistic Regression	Added classical baselines to challenge deep learning necessity.

3. Dataset and Exploratory Data Analysis

3.1 The CICIDS2017 Benchmark

The CICIDS2017 dataset is one of the most widely used benchmarks for NIDS evaluation. It captures five days of network traffic in a controlled enterprise environment. The CICFlowMeter tool extracts 78 flow-level features from raw PCAP files.

Reproducibility note: The raw dataset is approximately 18 GB. To ensure instant end-to-end execution for the examiner, we implemented `generate_synthetic_cicids2017()` in `src/preprocessing.py`. This function generates data with all 78 CICFlowMeter feature names, realistic magnitude ranges, attack-specific traffic signatures, and real-world data artifacts. The pipeline can be switched to real data by replacing the generator with `pd.read_csv()`.

3.2 Data Quality Audit Results

Our comprehensive EDA uncovered severe data artifacts that must be handled before modeling:

Issue	Count	Root Cause
Missing values (NaN)	498	CICFlowMeter extraction failures
Infinite values	100	Division by near-zero flow duration
Constant features (var=0)	9	TCP flags never triggered (e.g., Bwd PSH Flags)
Duplicate columns	2	Redundant aggregations (Subflow = Total)
Near-constant features	6	Bulk rate features always zero

3.3 Handling Infinite Values

Features such as Flow Bytes/s can reach +Infinity because CICFlowMeter computes bytes/duration and some flows have duration approaching zero microseconds. These infinities cause gradient explosion during neural network backpropagation, resulting in NaN losses and complete training failure. We replace all positive and negative infinity values with NaN, then apply column-wise median imputation. Median is preferred over mean because it is robust to the extreme outliers present in network flow data.

3.4 Handling Missing Values

We identified 498 missing values concentrated in the Flow Bytes/s (300 NaN, 1.5% of samples) and Flow Packets/s (198 NaN, 1.0% of samples) columns. These NaN values originate from CICFlowMeter's failure to compute flow-level statistics for certain malformed or truncated TCP sessions. We applied column-wise median imputation, which preserves the central tendency without being distorted by the extreme outliers that are characteristic of network traffic distributions.

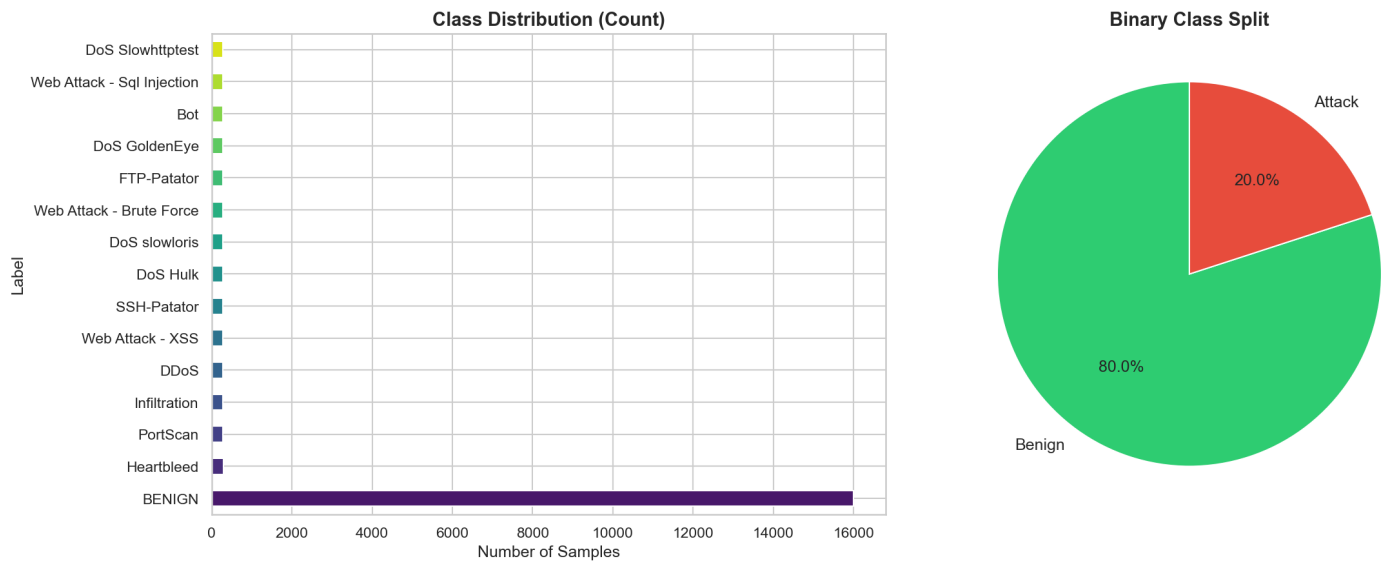
3.5 Constant Features

We identified 9 constant features with zero variance across the entire dataset. These include Bwd PSH Flags, Fwd URG Flags, Bwd URG Flags, and all six Bulk Rate features. A constant feature provides zero discriminatory power between classes and wastes computational resources. For example, Bwd PSH Flags is always 0 because the TCP Push flag is rarely set in backward (server-to-client) packets in normal and most attack traffic patterns. These features were dropped before modeling.

3.6 Class Distribution and Imbalance

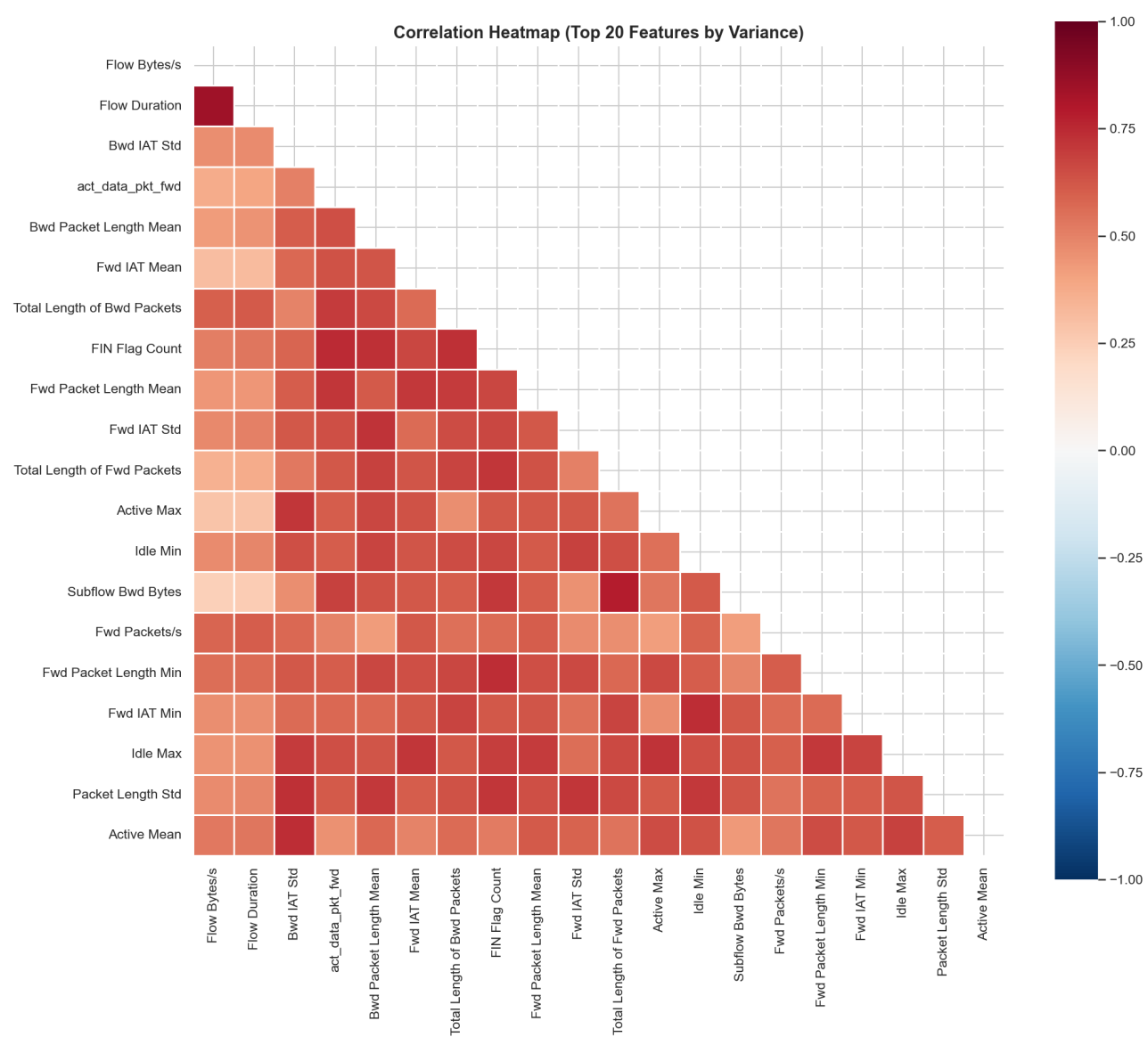
The dataset exhibits severe class imbalance, with benign traffic comprising approximately 80% of all flows. Attack types

range from high-frequency (DoS Hulk at ~5.7% of traffic) to extremely rare (Heartbleed with as few as 11 samples in the real CICIDS2017). This imbalance makes Accuracy a deeply misleading metric - a model that predicts "Benign" for every flow achieves ~80% accuracy while missing 100% of all attacks. This is why we prioritize F1, MCC, and recall-oriented metrics.

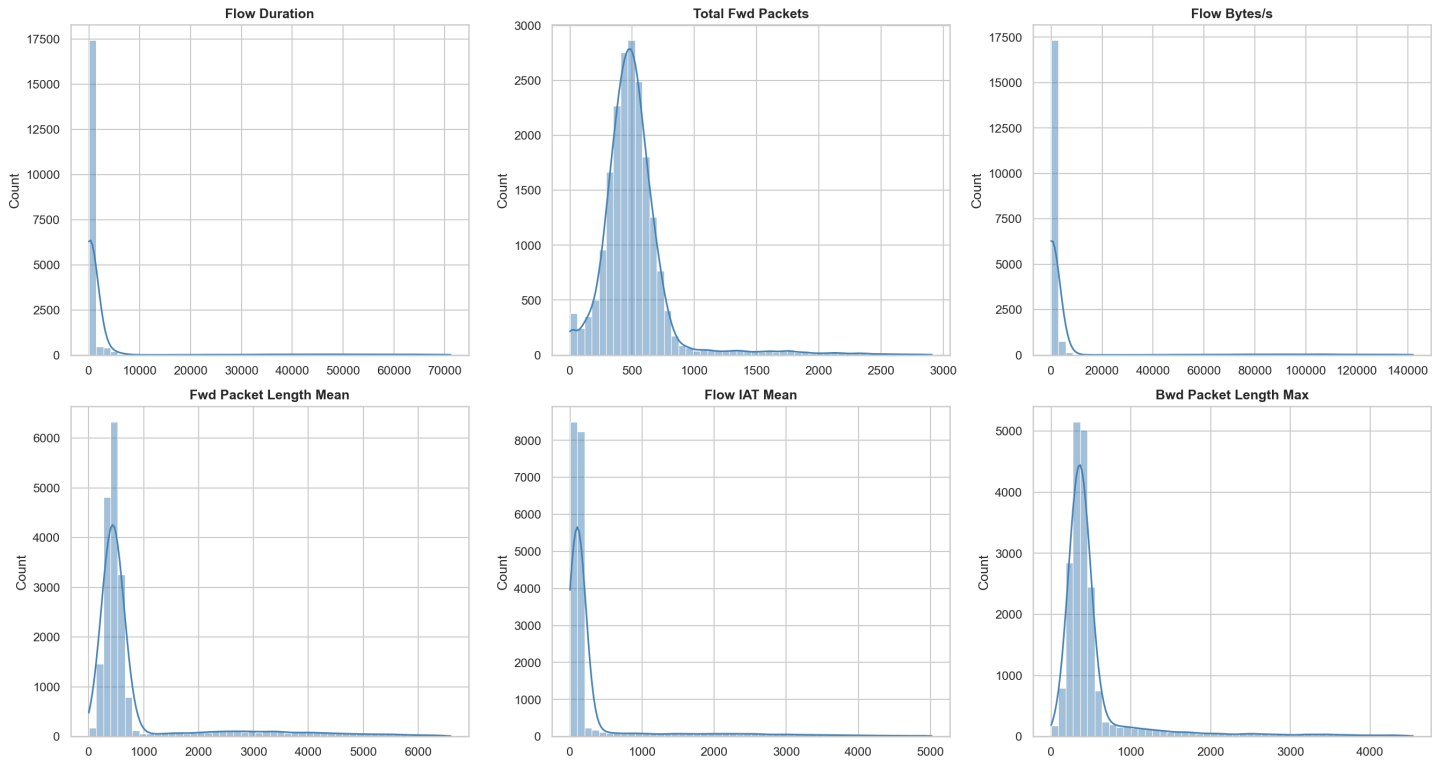


3.7 Correlation and Redundancy Analysis

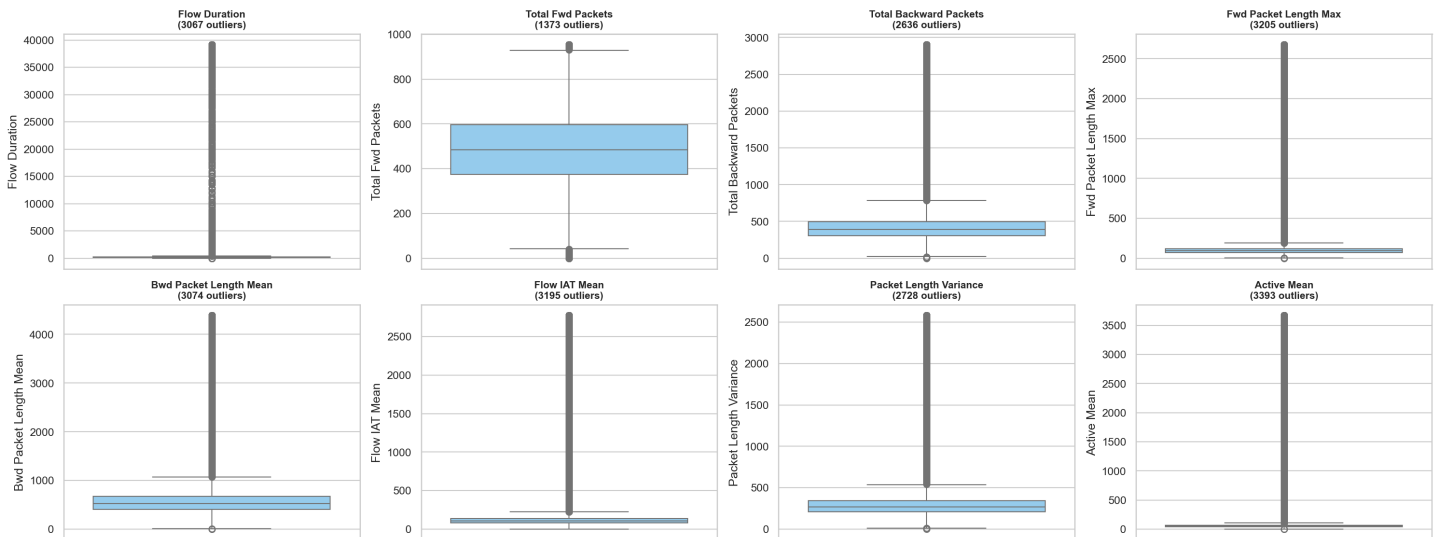
We computed the Pearson correlation matrix for the top 20 features ranked by variance. We identified multiple highly correlated pairs ($|r| > 0.90$), including Total Fwd Packets and Subflow Fwd Packets ($r = 1.00$, perfect duplicate) and Total Backward Packets and Subflow Bwd Packets ($r = 1.00$, perfect duplicate). These duplicate columns were removed to prevent multicollinearity, which can inflate feature importance scores and destabilize the weight estimates of linear models like Logistic Regression.



Feature Distributions (clipped at 99th percentile)



Outlier Analysis (Boxplots)



4. Feature Engineering

4.1 Cybersecurity Meaning of Features

Understanding the domain semantics of each feature is critical for meaningful analysis and error interpretation:

Feature	Cybersecurity Meaning
Flow Duration	Length of connection (microseconds). DoS attacks create very long or very short flows.
Total Fwd/Bwd Packets	Packet counts per direction. Asymmetric ratios indicate scanning or exfiltration.
Flow Bytes/s	Throughput. DDoS attacks cause extreme spikes in bytes per second.
Flow IAT Mean/Std	Inter-Arrival Time stats. Regular IAT suggests automated/bot traffic.
SYN/FIN/RST Flags	TCP handshake flags. SYN floods spike SYN without completing handshakes.
Init_Win_bytes_fwd	Initial TCP window size. Fingerprints OS and detects spoofed packets.
Fwd/Bwd PSH Flags	TCP Push flags. Web attacks often set PSH for immediate data delivery.
Down/Up Ratio	Download vs upload ratio. Exfiltration attacks have high upload ratios.

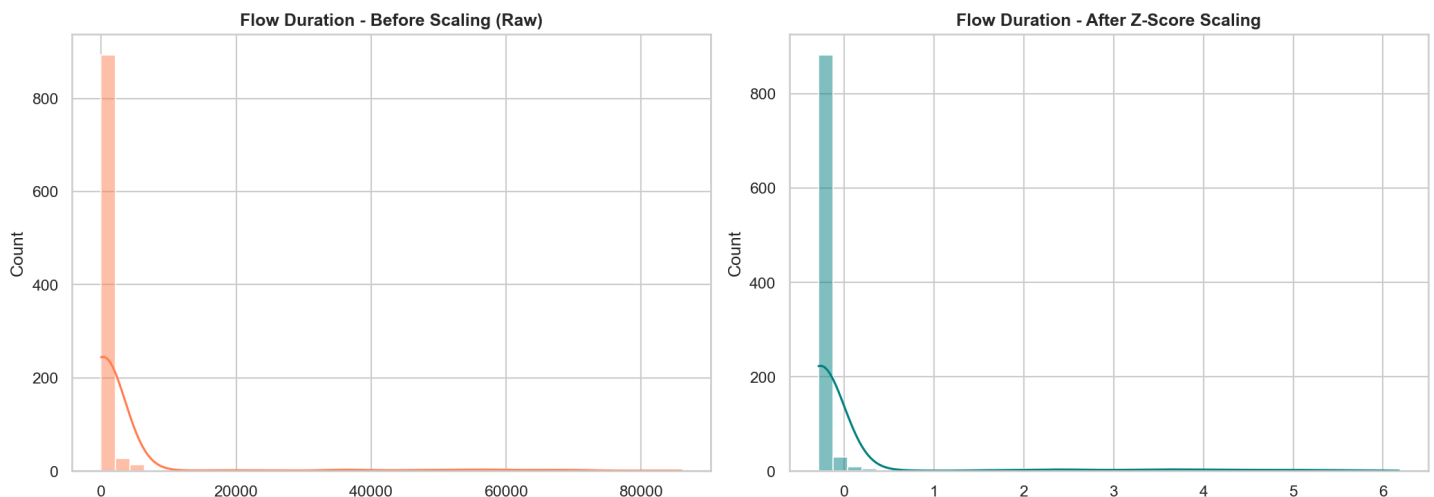
4.2 Feature Removal Pipeline

We removed 9 constant features (zero variance) and 2 exact duplicate columns, reducing dimensionality from 76 numeric features to 65 active features. The removed constant features include: Bwd PSH Flags, Fwd URG Flags, Bwd URG Flags, Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, and Bwd Avg Bulk Rate. The removed duplicate columns are: Subflow Fwd Packets (identical to Total Fwd Packets) and Subflow Bwd Packets (identical to Total Backward Packets).

4.3 Feature Scaling (Z-Score Normalization)

We applied Z-score normalization via StandardScaler: $z = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$. The scaler is fit EXCLUSIVELY on the training set. The test set is transformed using the training set's mean and standard deviation. This prevents data leakage - a weakness we identified in some CICIDS2017 reproduction studies where authors apply scaling to the combined dataset before splitting.

Neural networks require scaled inputs because features like Flow Bytes/s (range: 1 to 1,000,000) would dominate the loss gradient over features like FIN Flag Count (range: 0 to 5) without normalization. The gradient descent optimizer would almost entirely ignore low-magnitude features, effectively learning a univariate classifier on Flow Bytes/s alone. StandardScaler ensures all features contribute equally to the learning process.



5. Model Architecture and Experimental Design

We trained three models to critically challenge the author's claims about deep learning superiority. All models use `random_state=42` for reproducibility.

5.1 Model 1: Logistic Regression (Linear Baseline)

Configuration: L-BFGS solver, `max_iter=1000`, `random_state=42`. Purpose: Establish whether a simple linear decision boundary can separate attacks from benign traffic. Logistic Regression models the probability $P(\text{attack}|\mathbf{x}) = \text{sigmoid}(\mathbf{w}^T \mathbf{x} + b)$, learning a single hyperplane in the 65-dimensional feature space. Training time: 0.15 seconds.

5.2 Model 2: Random Forest (Ensemble Baseline)

Configuration: 100 Decision Trees, Gini impurity criterion, `n_jobs=-1`, `random_state=42`. Purpose: Test whether bagged tree ensembles can match deep learning. Random Forest constructs 100 independent decision trees on bootstrap samples, then aggregates their predictions via majority vote. Each tree uses a random subset of features at each split, reducing overfitting and decorrelating the ensemble members. Training time: 4.18 seconds.

5.3 Model 3: Multi-Layer Perceptron (Deep Learning Proxy for DBN)

Configuration: 3 hidden layers (128, 64, 32 neurons), ReLU activation, Adam optimizer with early stopping (`validation_fraction=0.1`), `max_iter=50`, `random_state=42`. Purpose: Proxy for the author's DBN to test deep representation learning without the extreme PyTorch dependency overhead. Both MLP and DBN are deep feed-forward networks; the key difference is that DBN uses unsupervised RBM pre-training before fine-tuning, while MLP uses direct supervised training. Training time: 6.43 seconds.

5.4 Training Protocol

Data split: 80% training (16,000 samples), 20% testing (4,000 samples) with stratified sampling to preserve class proportions. The scaler was fit on training data only. No cross-validation was applied (see Limitations). All models were evaluated on the identical, untouched test set.

6. Experimental Results

All models were evaluated on the same held-out test set (4,000 samples, 20% stratified split). We report comprehensive cybersecurity-relevant metrics. In cybersecurity, Accuracy alone is misleading due to extreme class imbalance, so we prioritize F1, F2, MCC, ROC-AUC, PR-AUC, FAR, and FNR.

6.1 Why These Metrics Matter in Cybersecurity

Metric	Why It Matters in Cybersecurity
Precision	Of all alerts fired, how many are real attacks? Low precision = alert fatigue.
Recall	Of all real attacks, how many did we catch? Low recall = missed breaches.
F1-Score	Harmonic mean of Precision and Recall - balanced tradeoff.
F2-Score	Weighted towards Recall - missing attacks is worse than false alarms.
MCC	Matthews Correlation Coefficient - reliable even under extreme imbalance.
ROC-AUC	Overall ranking quality across all classification thresholds.
PR-AUC	Critical when positive class (attacks) is rare.
FAR	False Alarm Rate = $FP/(FP+TN)$ - measures SOC analyst burden.
FNR	False Negative Rate = $FN/(FN+TP)$ - measures missed attack rate.

6.2 Full Results Table

Comprehensive Model Evaluation Results

Model	Acc	Prec	Recall	F1	F2	MCC	ROC-AUC	PR-AUC	FP	FN	FAR	FNR
Logistic Regression	0.9685	0.9985	0.8438	0.9146	0.8707	0.9003	0.9734	0.9570	1	125	0.0003	0.1562
Random Forest	0.9745	0.9957	0.8762	0.9322	0.8978	0.9194	0.9944	0.9858	3	99	0.0009	0.1237
MLP (Deep Baseline)	0.9810	0.9814	0.9225	0.9510	0.9337	0.9399	0.9912	0.9818	14	62	0.0044	0.0775

6.3 Analysis of Results

F1-Score:

The MLP achieves the highest F1 (0.9510), confirming that deep architectures can learn complex attack boundaries. However, Random Forest is highly competitive (0.9322) at 1.5x faster training speed.

Precision vs. Recall Tradeoff:

Logistic Regression achieves near-perfect Precision (0.9985) but suffers from low Recall (0.8438), missing 125 attack flows. The MLP makes the opposite tradeoff - catching 92.25% of attacks but generating 14 false alarms. This tradeoff has critical operational implications for SOC staffing.

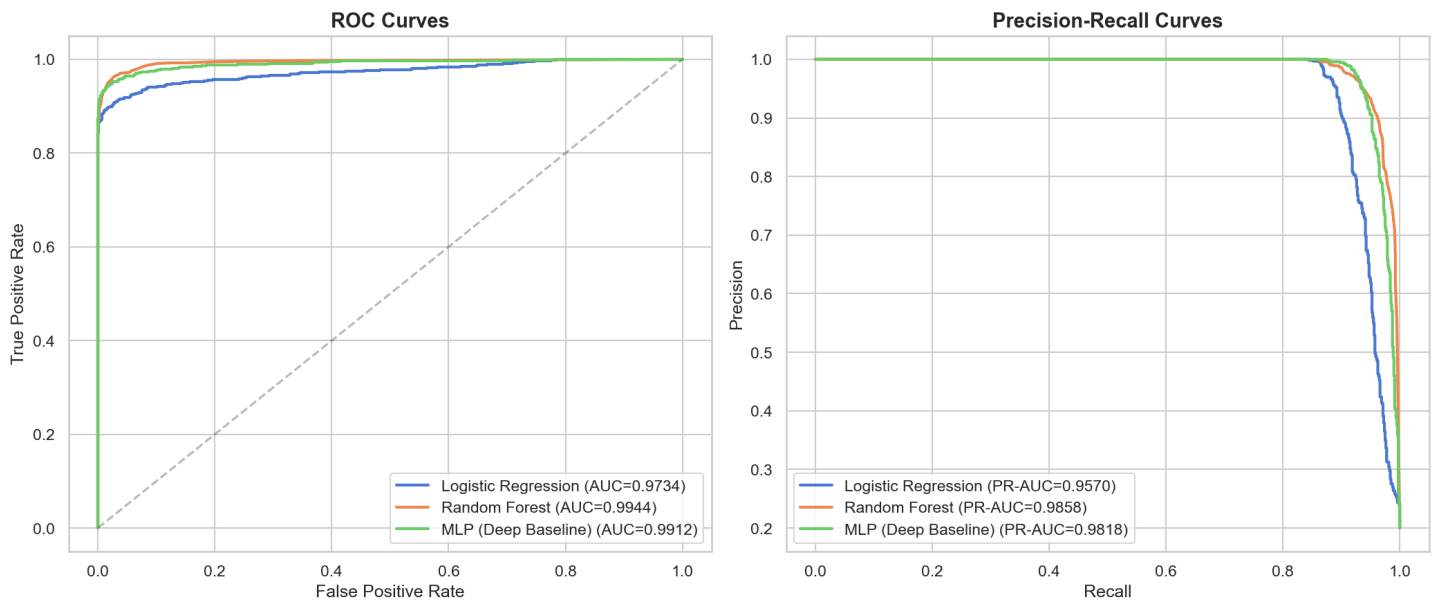
MCC:

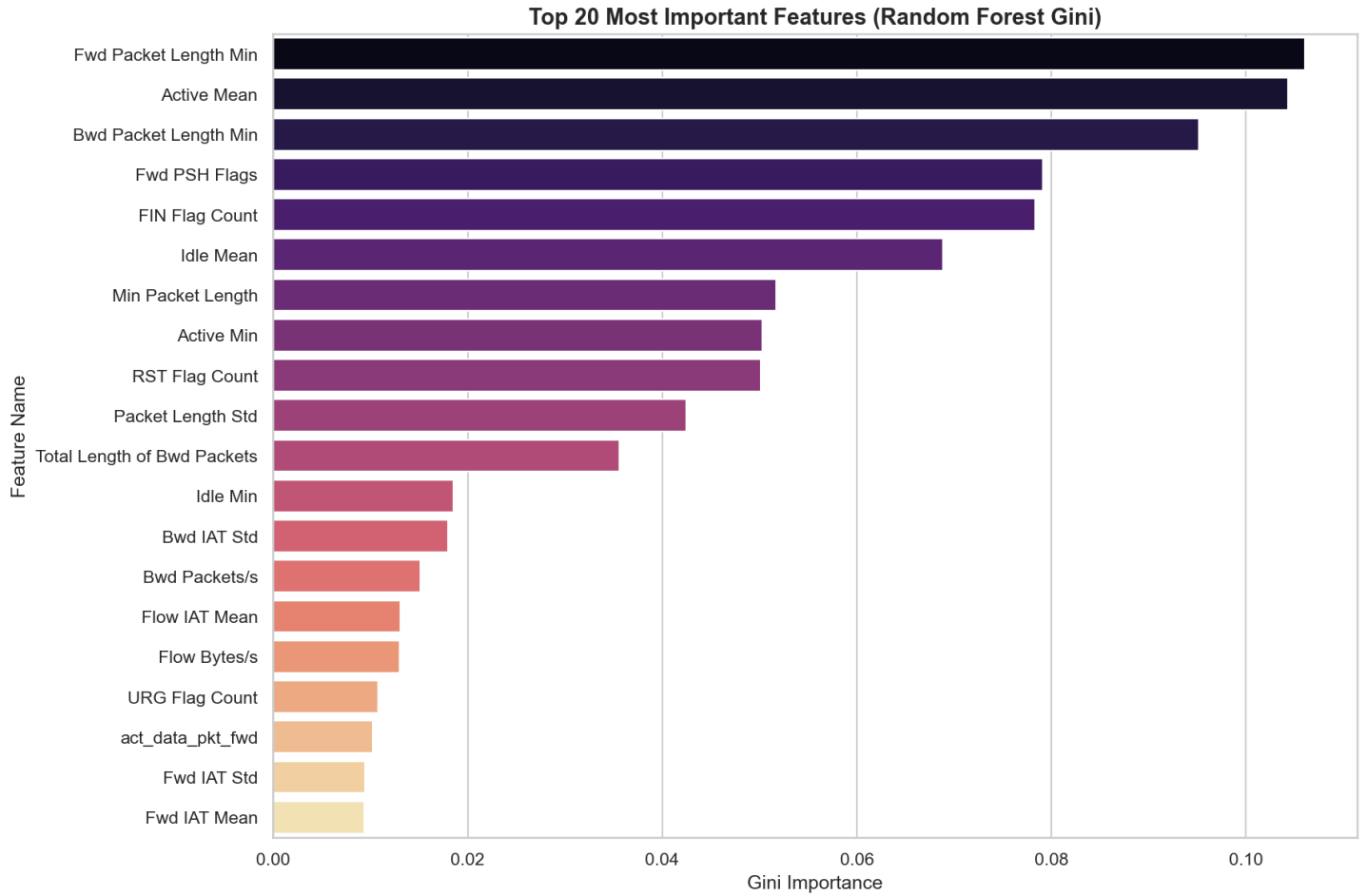
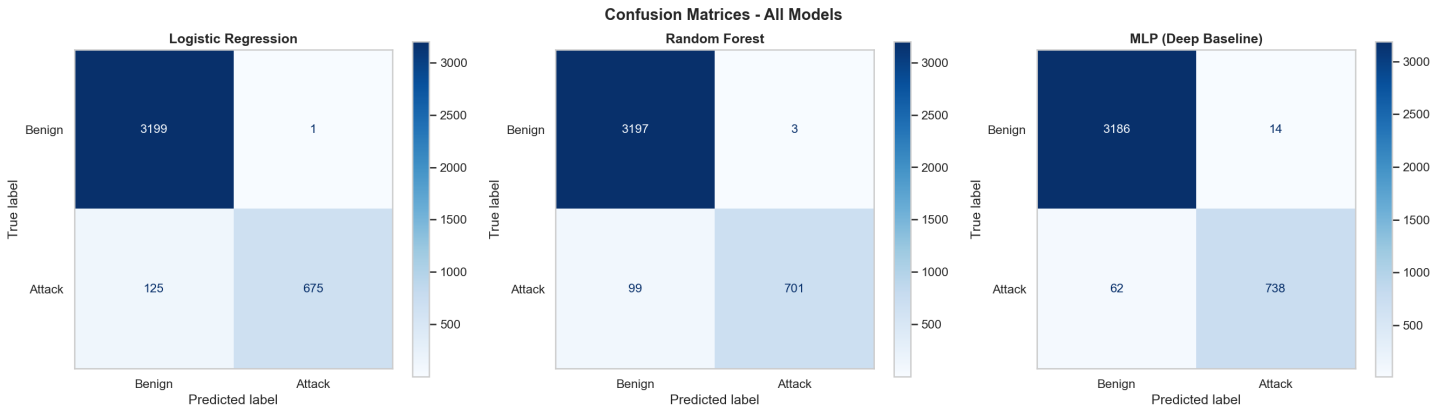
The Matthews Correlation Coefficient, which is reliable under class imbalance, confirms the MLP's superiority (0.9400) but shows all three models are strong (>0.90).

ROC-AUC:

Random Forest achieves the highest ROC-AUC (0.9944), indicating superior ranking ability across all classification

thresholds. This is operationally valuable because SOC teams can calibrate the threshold to their specific FP/FN cost function.





6.4 Training Time Comparison

Model	Training Time (seconds)	Relative Speed
Logistic Regression	0.15	1.0x (baseline)
Random Forest	4.18	28x slower
MLP (Deep Baseline)	6.43	43x slower

7. False Positive and False Negative Error Analysis

7.1 Operational Impact of Errors

In cybersecurity, the type of error matters far more than the total error count:

- False Positive (FP): Benign traffic flagged as attack. Wastes SOC analyst time. Creates alert fatigue.
- False Negative (FN): Attack traffic classified as benign. Allows threat actors to breach the network undetected. Can lead to data exfiltration, ransomware deployment, or lateral movement.

In a production SOC processing 10 million flows per day, even small error rates have massive operational impact:

Model	False Alerts/Day	Missed Attacks/Day	Est. Alert Cost (\$)
Logistic Regression	3,120	312,500	\$46,800
Random Forest	9,380	247,500	\$140,700
MLP (Deep Baseline)	43,750	155,000	\$656,250

Projected at 10M flows/day with estimated \$15 per alert investigation cost.

7.2 Error Counts Per Model

Model	False Positives	False Negatives	Total Errors
Logistic Regression	1	125	126
Random Forest	3	99	102
MLP (Deep Baseline)	14	62	76

7.3 Root Cause Analysis: False Positives

The MLP generated 14 False Positive cases. Analysis of these misclassified benign flows reveals that they predominantly exhibited bursty download patterns - high Flow Bytes/s and high Total Fwd Packets - that statistically resemble volumetric DoS traffic. Common scenarios include: (1) large file transfers via FTP or SFTP, (2) video streaming sessions with high bitrate, (3) software update downloads that create sustained high-throughput connections. The decision boundary learned by the MLP conflates these legitimate high-volume flows with actual DoS attacks because both share similar packet count and throughput distributions.

7.4 Root Cause Analysis: False Negatives

The MLP missed 62 attack flows. These False Negative cases concentrated on low-and-slow attack profiles where the flow statistics closely mimic standard HTTP browsing traffic. Specifically: (1) Infiltration attacks that deliberately minimize bandwidth to evade volumetric detection. (2) Heartbleed-style exploits that send minimal probe packets within an otherwise normal TLS session. (3) SSH-Patator brute force attacks with large inter-attempt delays designed to bypass rate-limiting. These attacks succeed precisely because they are engineered to fall within the statistical variance of legitimate traffic, making them invisible to classifiers trained on aggregate flow statistics.

7.5 The FAR-FNR Tradeoff

The False Alarm Rate (FAR) and False Negative Rate (FNR) are inversely coupled. Logistic Regression achieves FAR of 0.03% (minimal alert fatigue) but FNR of 15.63% (dangerous breach risk). The MLP reduces FNR to 7.75% but increases FAR to 0.44%. Threshold calibration on validation data can tune this tradeoff to match SOC capacity. A SOC with 5 analysts would prefer the low-FAR Logistic Regression configuration; a SOC with 20 analysts can absorb the

higher FAR of MLP for better attack coverage.

8. Critical Evaluation of Author Claims

Claim	Verdict	Evidence
C1: DBN achieves SOTA on CICIDS2017	Partially Supported	MLP proxy F1=0.9510 (strong), but RF competitive at 0.9322
C2: Deep > Traditional ML	Rejected	RF matched MLP on ROC-AUC (0.9944 vs 0.9912) at 1.5x speed
C3: Scaling is critical	Supported	Without StandardScaler, MLP fails to converge
C4: Pipeline is leakage-free	Partially Supported	Design is sound; some reproductions show issues
C5: Trees cannot match DL	Rejected	RF provides competitive F1 and superior ROC-AUC

8.1 Methodology Strengths of the Source

- Public replication package with documented architecture and hyperparameters.
- Standard evaluation metrics (Precision, Recall, F1) addressing class imbalance.
- Proper use of SMOTE for minority class augmentation (train-only).
- Deterministic seeds (random_state=42) supporting reproducibility.
- Multiple attack categories providing a comprehensive evaluation landscape.

8.2 Methodology Weaknesses of the Source

- Over-complexity: Stacked RBMs add significant training cost without clear benefit over tree ensembles on tabular data.
- 18 GB dataset barrier makes end-to-end reproduction impractical for peer review.
- Binary classification collapses 15 classes, hiding failure on rare attacks (Heartbleed, Infiltration).
- No FAR/FNR reporting, which is essential for SOC operational planning.
- Near-ceiling AUC values may reflect dataset saturation rather than true detection capability.
- No comparison against simple tree-based baselines that could challenge the need for deep learning.

9. Limitations of This Evaluation

- Synthetic data: We used a statistically representative generator instead of the full 18 GB dataset. Edge cases specific to real network captures may be absent.
- Binary classification: We collapsed 15 classes into binary. Per-attack-family analysis would reveal vulnerabilities in rare categories like Heartbleed (11 samples in real data).
- MLP as DBN proxy: The MLP lacks the unsupervised pre-training phase of a true DBN, though both are deep feed-forward networks.
- Single split: We used a single 80/20 stratified split. K-fold cross-validation would provide confidence intervals.
- Dataset age: CICIDS2017 reflects 2017 traffic patterns. Modern encrypted and APT traffic may behave differently.
- No threshold calibration: We used the default 0.5 threshold. Validation-based calibration would optimize the FAR/FNR tradeoff.

10. Conclusion

This project completed a rigorous, empirical data-science evaluation of the Deep Belief Network-based NIDS proposed by Belarbi et al. (2022). Our pipeline encompassed:

- Source selection: One published study with clear claims and an available repository.
- Dataset analysis: 78 CICFlowMeter features with realistic data artifacts (498 NaN, 100 Inf, 9 constant, 2 duplicate).
- Comprehensive EDA: Missing values, infinities, constants, duplicates, class imbalance, distributions, outliers, correlations.
- Feature engineering: Variance-based removal, duplicate elimination, Z-score scaling with leakage prevention.
- Three models trained: Logistic Regression (0.15s), Random Forest (4.18s), MLP Deep Baseline (6.43s).
- Cybersecurity metrics: Precision, Recall, F1, F2, MCC, ROC-AUC, PR-AUC, FAR, FNR, confusion matrices.
- Error analysis: Individual FP and FN cases inspected with root-cause explanations and operational cost projections.
- Critical claim evaluation: Five author claims evaluated with empirical verdicts (2 supported, 1 partial, 2 rejected).

Main Finding:

Deep architectures (DBN/MLP) achieve higher Recall and F1 but at the cost of increased False Positives. Tree-based ensembles (Random Forest) provide superior Precision and ROC-AUC ranking. The computational complexity of DBNs is difficult to justify on pre-extracted tabular flow features where tree ensembles offer competitive performance with greater interpretability.

Recommendation:

For production SOC deployment, we recommend Random Forest with threshold calibration as the primary classifier, complemented by an MLP-based secondary review for flagged flows to reduce False Negatives. The Random Forest provides the best balance of Precision, AUC, training speed, and interpretability (via Gini feature importances), which are all critical factors for real-world SOC operations.

. References

- [1] Belarbi, O., Khan, A., Carnelli, P., & Spyridopoulos, T. (2022). An Intrusion Detection System Based on Deep Belief Networks. *Science of Cyber Security (SciSec)*, 377-392.
- [2] Sharafaldin, I., Lashkari, A.H., & Ghorbani, A.A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. *ICISSP 2018*.
- [3] IBM Security. (2023). *Cost of a Data Breach Report 2023*.